# *SQL basic*

1.Create a New Database and  Table for Employees
Task: Create a new database named and Create a table named with the following column

| Column Name | Data Type | Constraint |
|---|---|---|
| Employee_id | INT | PRIMARY KEY |
| First_name | VARCHAR(50) | |
| last_name | VARCHAR(50) | |
| Department | VARCHAR(50) | |
| Salary | INT | |
| hire_date | DATE | |

- # SQL: Create a Database and Employees Table
- Use the following SQL commands to create a new database named `CompanyDB` and a table named `Employees` with the specified columns and constraints.
- ## 1. Create a new database
- CREATE DATABASE CompanyDB;
- ## 2. Select the database
- USE CompanyDB;
- ## 3. Create the Employees table
- CREATE TABLE Employees (
-  Employee_id INT PRIMARY KEY,
-  First_name VARCHAR(50),
-  last_name VARCHAR(50),
-  Department VARCHAR(50),
-  Salary INT,
-  hire_date DATE
- );
- ## 4. Verify the table structure
- DESCRIBE Employees;
- This command displays the column names, data types, and constraints of the `Employees` table.
- ## Explanation of columns

| Column Name | Data Type | Constraint / Description |
|---|---|---|
| `Employee_id` | INT | PRIMARY KEY (unique employee ID) |

| | | |
|---|---|---|
| `First_name` | VARCHAR(50) | Stores first name |
| `last_name` | VARCHAR(50) | Stores last name |
| `Department` | VARCHAR(50) | Stores department name |
| `Salary` | INT | Stores employee salary |
| `hire_date` | DATE | Stores employee joining date |

- Expected result: The `CompanyDB` database will contain an `Employees` table with six columns. The `Employee_id` column will uniquely identify each employee and cannot contain duplicate or `NULL` values.
- Note: These commands use MySQL syntax.

Q9. What is the Difference Between a Primary Key and a Foreign Key

A Primary Key uniquely identifies each record in a table, whereas a Foreign Key links one table to another by referring to a key in the other table.

### Differences

| Primary Key | Foreign Key |
|---|---|
| Uniquely identifies each row in a table. | Creates a relationship between two tables. |
| Must contain unique values. | Can contain duplicate values. |
| Cannot contain `NULL` values. | Can contain `NULL` values unless restricted. |

| | |
|---|---|
| Only one primary key constraint is allowed per table (it may contain multiple columns). | A table can have multiple foreign key constraints. |
| Ensures entity integrity. | Helps maintain referential integrity. |

Q10. What Are Constraints in SQL and Why Are They Used?

SQL constraints are rules applied to columns or tables to maintain the accuracy, validity, and integrity of data in a database.

### Types of SQL Constraints

| Constraint | Description | Example |
|---|---|---|
| `PRIMARY KEY` | Uniquely identifies each record and cannot contain `NULL`. | `Employee_id INT PRIMARY KEY` |
| `FOREIGN KEY` | Establishes a relationship between two tables. | `FOREIGN KEY (Department_id) REFERENCES Departments(Department_id)` |
| `NOT NULL` | Ensures a column cannot contain `NULL` values. | `First_name VARCHAR(50) NOT NULL` |
| `UNIQUE` | Ensures all values in a column or column combination are unique. | `Email VARCHAR(100) UNIQUE` |
| `CHECK` | Ensures values satisfy a specified condition. | `CHECK (Salary > 0)` |

| | | |
|---|---|---|
| `DEFAULT` | Provides a default value when one is not specified. | `Department VARCHAR(50) DEFAULT 'HR'` |

### Why Are Constraints Used?

1. Maintain data accuracy: Prevent invalid data from being entered.
2. Ensure data integrity: Keep database records consistent and reliable.
3. Prevent duplicate records: Use `PRIMARY KEY` or `UNIQUE`.
4. Maintain relationships: Use `FOREIGN KEY` to connect related tables.
5. Enforce business rules: Use `CHECK` to restrict values, such as requiring a positive salary.
6. Prevent missing values: Use `NOT NULL` for mandatory information.

-